Data Mining 1 (2026) 1–9 Submitted 12/06; Published 12/15

CS2 Round Outcome Prediction

Augustas Markevičius AUGUSTAS.MARKEVICIUS@MIF.STUD.VU.LT
Petras Radžiūnas PETRAS.RADZIUNAS@MIF.STUD.VU.LT
Titas Burinskas TITAS.BURINSKAS@MIF.STUD.VU.LT
Tomas Giedraitis TOMAS.GIEDRAITIS@MIF.STUD.VU.LT
Data Science and Bioinformatics study programmes
Faculty of Mathematics and Informatics, Life Sciences Center

Advisor: Jurgita Markevičiūtė

Abstract

Abstract abstract abstract abstract abstract abstract abstract abstract abstract abstract abstract abstract. Abstract abstract abstract abstract abstract abstract abstract. Abstract Chow and Liu (1968) abstract.

Keywords: A Keyword, Another Keyword, Third Keyword, Last Keyword

1. Introduction

Introduction introduction introduction introduction introduction, introduction introduction introduction introduction introduction introduction introduction introduction introduction. Introduction introduction introduction introduction (Pearl, 1988). Introduction introduction introduction introduction (introduction introduction) introduction introduction introduction introduction. Introduction introduction introduction, *introduction* introduction introduction.

1.1 Parameters [DRY]

Timeframe: 01.08.2026 - 11.10.2026

Teams: TOP 10 Between each other [HLTV]

Games: LAN

Maps: Dust2, Mirage

Snapshots: Every 5 seconds, (320 ticks)

Variables:

[In-Game] Side, Coordinates, Health, Round time, Win condition, Economy, Weapons, Utilities, Alive/Dead, X vs. X, C4 Position

[Meta] Teams, Players, Player ranking, Team ranking, History

Goal: Round outcome prediction based on position & other variables

H0: Round outcomes are random and independent on position & other in-game variables

H1: Round outcomes are predictable and dependent on position & other in-game variables

©2026 Augustas Markevičius, Petras Radžiūnas, Titas Burinskas, and Tomas Giedraitis.

2. Description

Our data mining research investigates a game of Counter-Strike 2 (CS2 onwards) by trying to predict the outcome of the round based on player's position on the map and other important factors like side, health, weapons, team economy, etc. A single map is played until one of the teams win 13 rounds out of 24, or in case of a tie, 12-12, they play additional tie-breaker rounds. There are two sides (also called teams) in each match, CT and T. Each map is played differently due to different geometry that produces unique game situations, timings and optimal positions for each side, so in order to predict a round-win outcome all of this must be taken into consideration and can't translate from map to map and must be evaluated for each map separately.

The core premise of why researching CS2 is particularly intresting is because the players in the game itself operate on limited knowledge under certain fog-of-war and intrisically perform probabilistic estimations about the intentions and the positions of the enemy team. While dueling is a completely mechanical skill, and may take a second to end, estimations about enemy positions and optimal ways to counter them are purely informational and take up the majority of the game. So probabilistic models might shine light not only about some extrinsic qualities of the game, like what factors determine win outcomes the most, but also give insights into how the game is played in itself. Other games also have similar qualities like fog-of-war or prediction of the opposing team's intentions, however, CS2 is unique in the fact that due to it's digital nature, every event and facet of the game is stored as information and can be extracted as data and reproduced near-verbatim, making it a desirable medium for data mining.

In the game there are two distinct sides, CT (Counter-Terrorists) and T (Terrorists). Round win conditions for each side are different. For Ts it's 1. Planting the bomb at Bombsites and successfully running out the timer of the bomb, 2. Eliminating all CTs. For CTs it's 1. Defusing the planted bomb, 2. Eliminating all Ts, 3. Running out the round timer. Due to this dynamic, Ts are considered and attacking team, and CTs the defending team. Round win-rates are usually more skewed towards CT side on most maps, while some retain equal side win-distribution [optional: add statistic here].

For our research, we chose to investigate professional games of the Top 10 teams playing each other based on hltv.org CS Rating 3.0 which is, if not one of the most robust rating systems, at least the most widely accepted and used. Hltv.org database evaluates whether the teams at the time of their match were in Top 10 and filters out only these results. Of all types of matches, only LAN (Local Area Network) tournament matches were selected. The timeframe ranges from 2026.08.01 to 2026.10.11, during which 60 overall matches were played, of which great majority of them were BO3 (Best of 3), BO5 rarely, and BO1 once. During these matches, 150 maps overall were played. We've used convenient in-built data filtering options in hltv.org/stats for queries and slicing.

We chose two of the most played maps, Dust2 and Mirage, which, out of 7 maps in the map pool, were played by Top 10 teams 30 times each during set period, which is 40% of the time. Out of all 60 matches, one of these two maps were played 33 times, both of these maps 14, and 13 matches where neither of them were played. Altogether, out of 60 matches, 47 times at least one of the maps were played, which is 78.33%. Other than the popularity, there were other reasons that these two maps were picked that we will go down into below.

Dust2 CT/T round win-rate distribution is 49.7%/50.3% respectively, almost an ideal 50-50 "coin-toss", so to say, and Mirage has greater variance, with 55.8%/44.2% distribution. We find this difference an interesting point of investigation, by trying to see how our model will fare for maps which have side-oriented and side-independent round outcomes. These two maps are well-known in pro-play and have been played for decades, so win-rate distribution isn't a sort of recency-bias, but a well set-in "quality" about the map itself. While analyzing all maps would've been ideal, only two maps were picked because of the sheer amount of data it might've produced and the requirements to handle it all properly, we assumed a more leaner approach that might still investigate the differences between maps.

The reason why above mentioned parameters were chosen is because all of them are expected produce a greater consistency in data, while also removing uninformative randomness. The zero hypothesis of this research is that round outcomes are random and don't depend on positions and other in-game factors, therefore it is required to remove out-of-game random factors as much as possible in order not to conflict the two. As with all games, there's an element of randomness, which must be controlled in order for a game to be considered competetive (i.e. skill-based) and predictable. It is a somewhat desirable quality for a game or a sport to be predictable "on-paper", where quantitative features inform the outcome of the game, and statistics are often used for this reason.

In CS2, the professional play in general terms is more consistent by nature than public games because of regular team roster, high individual skill, etc. Top 10 teams have greater parity with each other and consistently outperform teams outside Top10. Top 10, no matter the period, are a benchmark of of 'how the game should be played' at the highest skill-level. Same goes for matches played in LAN tournaments as opposed to On-Line games, where match outcomes are less predictable due to poorer connectivity quality and other factors which are harder to define quantitavely, like motivation (due to lesser stakes). A short timeframe of 72 days was chosen due to the ever-changing nature of competetive video games, where some updates might alter the game more drastically. This timeframe also coincides with a recent update to the map-pool, where Cache was added and Overpass was removed, the games start right after a summer player break that followed a Major tournament (IEM Cologne 2026). The recency of the data was chosen more for relevance reasons rather than unique set of quality within the data itself, as in, the findings might be of more importance in the field rather than the findings from data gathered from a few years ago. It should be taken into account that despite all this, some changes within the game, team rosters, tactics, etc. do occur over time and cannot be avoided completely. All of these decisions can be backed more robustly with various separate analyses, but it falls outside the scope due to the constraints of this research and should be considered as expert curation that comes from many years in the field. This doesn't not mean that it shouldn't be questioned or that it is immune to critique.

In general, many of the decisions were also driven by the fact that our approach will require ML training so we had to narrow it all down before we do that step, so as to not get overwhelmed by sheer amount of data that it might produce, as we assume it might be quite a lot (for instance, in an initial data test, a single map produced 1,600,000 rows of unfiltered tabular info). It is assumed that the amount of possible interactions between variables will be extremely high, and making a dataset more narrow will make it more manageable for the intended scope of our research.

3. Literature Review

One of the key challenges to initialize a relevant database was to parse the Demo files that store the entire information about the game tick-by-tick. Tickrate is the rate of recorded instances per second. Actual matches have 128 tickrate, Demo files have 16 tickrate to save space. While this may incure some loss of information, 16 instances per second is considered enough to account for all of the important events, except micromovements or minuscule information about crosshair placement or accurate hitbox hit instances. We primarily used demoparser2 Demo parser `https://github.com/markus-wa/demoinfocs-golang` to deal with the challenge of translating raw files into usable data. It is one of the most widely used tools in the scene for these exact application. Its features include tracking game events (kills, shots, round starts/ends, footsteps), game state (players, teams, grenades, ConVars), grenade projectiles/trajectories, and various other functions which are less important for our research. One of the stated top priorities of demoparser2 developers are performance and concurrency, making it a great option if scaling our project should ever be considered. Another tool that we used by way of functionality extension rather than substitution of demoparser2 was Awpy `https://github.com/pnxenopoulos/awpy,` which was programmed on Rust and built on top of the original demoparser2 and is a port to Python programming language to which our group members have a greater affinity. Another reason why Awpy was relevant is that while, at least on the surface, it gives less-control, it may provide a more clearer and ready-made functionality which is convenient for quick review of a given Demo file. Awpy doesn't require writing a custom code for it, making it a fast inspection tool. However, one has to keep in mind that these tools are not that different and choosing one over the other comes down to preference.

As for the CS2 round prediction models, there are many varying kinds, hence evaluating them is critical, and some might be deemed a good baseline to kick off our research. One of the most similar in goals and structure is `https://github.com/TaiZo1/cs2-win-prediction,` a data pipeline, exploratory analysis and classical modeling project which uses demoparser2 and Awpy, scrapes and compares team statistics of Top 5, Top 10, Top 20, Top 30 and Top 50 teams (according to HLTV.org) across Mirage, Inferno, Dust2, Nuke, Ancient, Train, Anubis, Overpass, or global (all-maps aggregate). Snapshots were taken at freeze_end + 2s (after all buys, before any duel). Its model trained three logistic regressions, split into pistol, post-pistol and normal rounds as distinct regimes of the game, "with a temporal train/test split grouped by match (the most recent 20 % of maps form the test set: 302 maps from November 2025 onwards, i.e. 599 pistol, 601 post-pistol and 5,376 normal rounds). Four nested feature configurations are compared (A = round-state only, B = +HLTV, C = +map, D = + quasi-target features for control)." Three key takeaways that have potential for testing with our own research are that "1. The round type is the strongest predictor of predictability, pistol rounds stay barely above chance; post-pistol rounds are almost deterministic. 2. The immediate round state is the dominant signal. Adding HLTV stats or map identity gains at most +0.01 AUC over the round-state baseline on normal and post-pistol rounds and 3. PCA does not yield a useful dimensionality reduction. 12 components (out of 28 to 43 input features) are needed to capture 80 % of inertia in every regime."

One of them is https://github.com/ratulsarker/CS2_RoundPrediction which "Predicts which team (CT or T) will win a given round in CS2 using deep learning on mid-round game state snapshots." At the time of writing, since there is not stated license, and we didn't get confirmation if we can reuse the project's material for our purposes, we may only use claimed results as a benchmark and infer the efficiency of methods that were used. Their research has shown that MLP Tuned (Optuna, 50 trials) has shown best results [Test accuracy - 95.75%, AUC-ROC 0.9945] with MLP Standard (86 features) coming second [93.86%, 0.989] and KNN (k=3) coming in third [93.79%, 0.978], with Random Forest (500 Trees), XGBoost (86 features), Logistic Regression and LSTM (round-start, 15 features) as runners-up in that particular order. Research claims that top predictive features are 1. player count differential man_advantage (29.8%), 2. equipment_advantage (8.4%), 3. health_advantage (4.7%), 4. and t_allive, kills_this_round_t, armor/helmet_counts (percentage not declared). Their analysis involved 530 matches, 9,746 rounds and 161,271 snapshots, with 86 features extractedd from mid-round game state, and class balance of 48.9% CT wins / 51.1% T wins.

There's a very recent (last update 18-10-2026) https://github.com/wyn0n0/CSMapWinPredict project that has a very similar scope of analyzing Demo files with an additional in-built radar-type 2D visualizer, and a target of predicting current round winner updated about once per second. One of it's key advantages is the live demo replay function itself, where one can see win-rate probability as the replay goes, which is very useful for testing. However, there are limitations. "Predictions and map settings currently focus on Mirage. Results on other maps have not been verified. Large Demos may require substantial processing time and memory." The fact that it's based on Mirage serves our research, though only partially and some reservations have to made. Slow processing time would become a liability if scaling a solution would ever be considered, and our project prefers to have that option in case we need a greater dataset.

One of the more lean yet effective round outcome predictor projects is https://github.com/d-roho/CSGOPredictor, and while recent (made in 2025), it is made for an older iteration of the game - CS:GO, rather than CS2. One of it's key advantages is that it can do live round predictions. Program used Logistic Regression model, and robust, verified and prepared dataset of 700 demos for model training gathered from "high level tournament play in 2019-2020 ... warmup rounds and restarts have been filtered ... a round snapshot has been recorded every 20 seconds until the round is decided. The total number of snapshots is 122411. Snapshots are i.i.d and should be treated as individual data points, not as part of a match." Project has data analysis documentation that other sources unfortunately often lack which can be checked out here https://www.kaggle.com/datasets/christianlillelund/csgo-round-winner-classification. It is one of the more used datasets for many similar projects and comparisons of results and methods can readily be made between them.

There are a few deep-learning solutions to the CS2 round prediction problem that are worth mentioning and comparing, despite the fact that DL falls outside our intended methods. https://github.com/Gary2005/cs-net is transformer-based path prediction & game-state forecasting for Counter-Strike 2 trained with Pro architecture (d_model=768, 138.7M params). What it claims to achieve is predict player's movement trajectory, along with a spatial-only downstream win-rate prediction that aligns with our own aim that uses 4 second snapshot windows, making an interesting example for comparison between DL and

ML solutions. However, the detailed documentation is in Chinese, so correct reproducibility may be unreliable because of translation. Another neural network solution comes from `https://github.com/eigenpaul/cs2-evalbar`, a "roughly 70K-parameter GraphTemporalTransformer over a 20-second, 4 Hz state window." While claiming meaningful results by game phase (<15s, 30-60s >=90s) with accuracy of 0.6, 0.68, 0.88. Another interesting finding is that the project claims "extra capacity overfits near 95 groups, but widening can help once the dataset reaches 142 groups." One big caveat is that the application was trained on Inferno, another map, so the results will be hard to compare.

Xia et al. [9] focused on round win and match win predictions using XGBoost, Logistic regression and Decision tree ML methods. While the data was limited to CS:GO professional matches from 2022 and only the matches happening in the Dust2 map, the results showed that both XGBoost and Logistic regression could be used to effectively predict match wins with approx. 86% accuracy, and player death in a given time window with approx. 66% accuracy. Key predictors included economic factors, spatial positioning and team coordination. Additionally, awpy was used to parse match demo files for feature extraction.

While not directly relevant to our goal, Björklund et al. [10] used a neural network approach to predict CS:GO match outcomes. The interesting part of the work is that rather than using player data for direct prediction, instead the data was clustered into groups which were intended to represent unique play styles. These groups would then be used to evaluate the composition of opposing teams and that evaluation used to predict the match outcome. While the accuracy of the model was modest, being around 61 – 65%, conceptually this work is of interest because it demonstrates that team composition based on its players' play-style can be used as a metric for match outcome prediction.

Jing et al. [11] have used machine learning methods (primarily XGBoost) along with SHAP for intrepretation to distinguish key performance metrics in CSGO and CS2 professional players that correlate with a higher HLTV player rating. While this study is very valuable in determining said performance characteristics, it is limited by the fact that the features used were highly correlated and that the response variable is a HLTV rating which has been turned into a binary value representing "good" and "okay" players. Furthermore, the rating formula in the CSGO era prioritizes fragging while the CS2 era formula is unknown, providing a high level of uncertainty as to whether the chosen features truly correlate with performance, e.g. with higher probability of winning a round or a match. Other metrics such as player past performance or role are also not accounted for. This is merely to say that there is room for improvement with regards to our set goals.

4. Data

4.1 Data set

The data set used is ...

cell1	cell2	cell3
cell4	cell5	cell6
cell7	cell8	cell9

Table 1: Example table.

Some more text...

4.2 Exploratory data analysis

Exploratory data analysis was performed ...

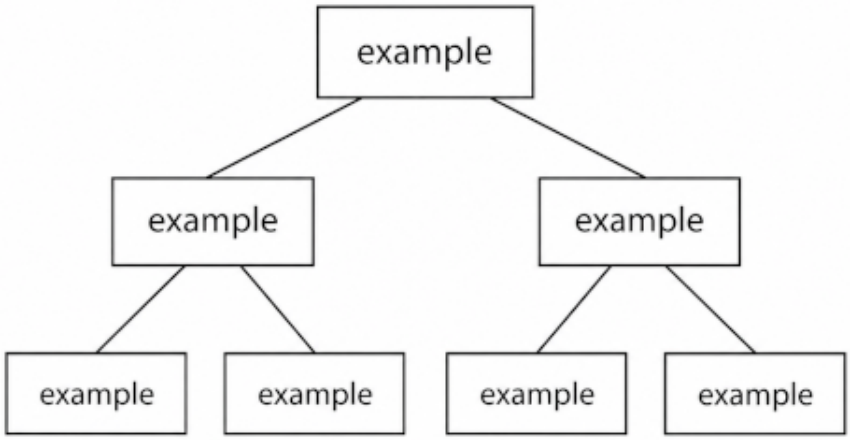


Figure 1: Example image. Source: Romero (2020)

4.3 Methodology

possible training models

Logistic Regression

KNN

XGBoost

MLP

LSTM

Sci-kit learn

`https://github.com/markus-wa/demoinfocs-golang` [Go Demo-parser]

`https://github.com/pnxenopoulos/awpy` [Python-based demo data analysis]

`https://github.com/ratulsarker/CS2_RoundPrediction` [Deep-Learning, a good example]

`https://github.com/Gary2005/cs-net` [Transformer-based round win prediction, predicts movement]

`https://github.com/wyn0n0/CSMapWinPredict` [Replay and prediction with a display]

`https://github.com/eigenpaul/cs2-evalbar`

`https://github.com/kamalchah/CounterStrikeWinProb`

https://github.com/TaiZo1/cs2-win-prediction

1. go parser that allows to collate data from demo files based on events. Oriented towards efficiency and speed.

2. similar to go parser and based on go parser, does splicing and other functions with different semantics, creates dataframes, etc. Perhaps less efficient but more user-friendly, has some features that go parser doesnt. [check what specifically]

The question of "Why these two? Why not one or the other?"

3. CS2_RoundPrediction. Uses Deep-Learning methods. Unknown license, so for now might only inspect. Can't use deep learning in our project.

4. Cs-net: Transformer-based path prediction & game-state forecasting for Counter-Strike 2. 3 key parts: path prediction, spatial-only models, 3D visualizer. Has pretrain functions. Uses 4 second windows. Documentation in Chinese, so unreliable information through translator.

5. CsMapWinPredict: CS Demo Map is a Counter-Strike 2 replay and win-probability analysis project. It turns .dem recordings into an interactive radar replay, showing match progress alongside round-win estimates based on player status, equipment and the bomb situation. The model uses information available at the current point in the recording, with estimates usually updated about once per second. The prediction target is the current round winner, not the final result of the map or match. Last update 2026-10-18, Mirage-only.

6. A CS2 round-probability and demo-analysis toolkit. It combines a gradient-boosted baseline with a temporal graph-transformer model, probability calibration, and a web-based analysis interface. It currently focuses on Inferno.

7. A CS:GO round-win probability pipeline that specifically investigates player-position embeddings. This is conceptually relevant to your spatial-feature goal, although it targets the older CS:GO dataset and may require adaptation for CS2 demos.

8. A professional-match round prediction pipeline with demo parsing, feature engineering, and logistic regression. It predicts from the freeze-time state, making it useful as a baseline, but it doesn't capture the full evolution of a round.

9. X. Xia, A. Salinas, and F. Morstatter, "Precision under fire: Analysis and predictive modeling in Counter-Strike: Global Offensive," IEEE Conference on Games (CoG), pp. 1–8, Aug. 2025, doi: 10.1109/cog64752.2025.11114164.

10. "Predicting the outcome of CS:GO games using machine learning," BACHELOR OF SCIENCE THESIS, CHALMERS UNIVERSITY OF TECHNOLOGY, 2018. [Online]. Available: https://odr.chalmers.se/server/api/core/bitstreams/e919ca56-3995-46b1-9546-a09b83f8b content

11. S. Jing, M. M. Awang, W. A. M. W. Pa, P. Pan, D. Memmert, and Z. Zhang, "Interpretable machine learning with SHAP for esports performance analysis of professional counter-strike players from 2012 to 2025," International Journal of Sports Science & Coaching, vol. 21, no. 5, pp. 3298–3314, Oct. 2025, doi: 10.1177/17479541251388864.

12. Y. Eshrah and Y. M. I. Hassan, "CS2AnalyticsAPP: A Multi-Player Shooter Game Analytics Framework," 2025 35th International Conference on Computer Theory and Applications (ICCTA), Alexandria, Egypt, 2025, pp. 119-124, doi: 10.1109/ICCTA68914.2025.11519898.

5. Results

The results achieved were ...

6. Conclusions

These conclusions follow ...

Acknowledgments

We would like to thank...

References

C. K. Chow and C. N. Liu. Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, IT-14(3):462–467, 1968.

Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufman Publishers, San Mateo, CA, 1988.

Stephanie Romero. The 3 most important data mining techniques. `http://annettesabode.com/the-3-most-important-data-mining-techniques/`, March 2020.

Appendix A.

In this appendix we show that...